

A Radix-4 Booth Multiplier in Structural VHDL

Architecture, optimization and synthesis at 32 bits in Nangate 45 nm

Erfan Taheri

Nangate 45 nm · Synopsys Design Compiler · ModelSim

What was built

A signed 32×32 radix-4 Booth multiplier in structural VHDL, with three architectural optimizations applied in sequence and measured under an identical synthesis flow.

Variant	min. achieved	min. area	gap closed
Wallace, sign-extended (baseline)	1.86 ns	6346 μm^2	—
+ sign-extension elimination	1.87 ns	5975 μm^2	19.5%
+ Dadda reduction	1.78 ns	5620 μm^2	38.1%
+ fused selector	1.76 ns	5029 μm^2	69.1%
Inferred $A * B$ (reference)	1.63 ns	4441 μm^2	reference

6 configurations $\times$ 21 clock periods = 126 synthesis runs. Every variant verified exhaustively at 8 bits before synthesis at 32.

Single source tree.

Every variant is an alternative architecture of an existing entity, selected by a VHDL configuration. No file is duplicated.

Absolute reference.

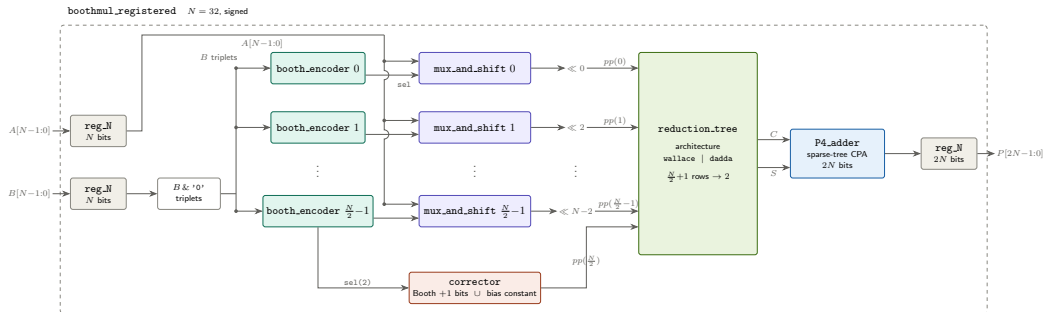
A design consisting of $P \leq A * B$ is synthesised with the same script, constraints and library, so results are not measured against an arbitrary baseline alone.

Verification before measurement.

A variant is synthesised only after it passes the self-checking testbench exhaustively at 8 bits. Area and timing reports do not distinguish a correct design from an incomplete one.

1. Derive the optimization analytically.
2. Implement it as a new architecture.
3. Verify exhaustively at $N = 8$.
4. Sweep 21 clock periods at $N = 32$.
5. Reconcile the measurement with the prediction.

Architecture



Radix-4 modified Booth recoding

Overlapping triplets of B — bits $(2i+1, 2i, 2i-1)$, with $B_{-1} = 0$ — map to a signed digit $d_i \in \{0, \pm 1, \pm 2\}$:

triplet	d_i	row
000, 111	0	zeros
001, 010	+1	$+A$
011	+2	$+2A$
100	-2	$-2A$
101, 110	-1	$-A$

$$A \cdot B = \sum_{i=0}^{N/2-1} d_i A \cdot 4^i$$

$N/2$ rows instead of N . The reduction tree dominates area and delay, and its cost grows with the row count, so halving the rows is the point of the recoding.

Multiplying by two is a wired shift and costs nothing.

Negation is not free. It is the origin of two of the three optimizations.

Optimization 1 — the cost of the sign extension

Row i holds $m_i = d_i A$, which fits in $N + 1$ bits of two's complement. Placed at weight 2^{2i} in a $2N$ -bit array and summed as an unsigned quantity, bit N must be replicated up to position $2N - 1$.

Replicated bits in row i : $N - 1 - 2i$. Across the array:

$$\sum_{i=0}^{N/2-1} (N - 1 - 2i) = \frac{N}{2}(N - 1) - \frac{N}{2}\left(\frac{N}{2} - 1\right) = \boxed{\frac{N^2}{4}}$$

N	total array bits	sign bits	fraction
8	56	16	28.6%
16	208	64	30.8%
32	800	256	32.0%

At $N = 32$: 256 of 800 partial-product bits carry no information beyond the single bit they replicate. Each must still be generated, routed and compressed.

Optimization 1 — the inversion identity

For a two's complement number with sign bit s :

$$m_i = -s \cdot 2^N + \sum_{j=0}^{N-1} m_{i,j} 2^j$$

Since $s + \bar{s} = 1$ for a single bit, $-s = \bar{s} - 1$, hence

$$-s \cdot 2^N = \bar{s} \cdot 2^N - 2^N$$

Define the *stored* row — low N bits unchanged, sign bit inverted, zeros above:

$$R_i = \bar{s} \cdot 2^N + \sum_{j=0}^{N-1} m_{i,j} 2^j \quad \implies \quad \boxed{m_i = R_i - 2^N}$$

The sign extension becomes one inverted bit. What remains is a constant error of exactly 2^N per row — independent of A , of the Booth digit, and of the row index. Not an approximation.

Optimization 1 — the correction constant

Summing the per-row bias over the array:

$$E = \sum_{i=0}^{N/2-1} 2^{N+2i} = 2^N \cdot \frac{4^{N/2} - 1}{3} = \frac{2^N(2^N - 1)}{3}$$

The datapath computes in $\mathbb{Z}/2^{2N}\mathbb{Z}$, so the correction is simply the additive inverse:

$$C = 2^{2N} - \frac{2^N(2^N - 1)}{3} = 2^N \left(2^N - \frac{2^N - 1}{3} \right)$$

N	C , bits $2N-1$ down to N
4	0xB
8	0xAB
16	0xAAAB
32	0xAAAAAAB

C is a multiple of 2^N : its lower N bits are zero.

For even N , $(2^N - 1)/3 = 0101 \dots 01$, so 2^N minus it gives $1010 \dots 1011$. The repeating pattern comes from the factor $1/3$.

Optimization 1 — why it costs no extra row

Booth already needs one extra row for the $+1$ that each one's-complement negation owes. That row occupies **even positions of the lower half** only. The constant C occupies the **upper half** only. They share one row without collision.

```
architecture no_sign_extend of corrector is
    constant SE_CONST : std_logic_vector(2*N-1 downto 0) := sign_ext_const(N);
begin
    gen_neg: for i in 0 to N/2-1 generate          -- low half: Booth increments
        correction_v(2*i)    <= neg_bits(i);
        correction_v(2*i+1) <= '0';
    end generate;
    correction_v(2*N-1 downto N) <= SE_CONST(2*N-1 downto N);  -- high half
end architecture;
```

The high half is a tie to the supply rails: no logic at all. C is a $2N$ -bit quantity, too large for the VHDL integer type at $N = 32$, so `sign_ext_const` builds the pattern bit by bit at elaboration time.

Optimization 1 — measured

	baseline	optimized	change
Partial-product bits	800	561	−29.9%
Total cells @ 3.0 ns	4639	4455	−4.0%
XOR + XNOR @ 3.0 ns	1346	1332	−1.0%
Area @ 3.0 ns	6501 μm^2	6196 μm^2	−4.7%
Minimum area	6346 μm^2	5975 μm^2	−5.8%
Minimum achieved period	1.86 ns	1.87 ns	+0.5%

29.9% **fewer bits**, 5.8% **less area**. The removed bits are constant-valued sign replications, and constant propagation already eliminates most of the logic that would process them.

What the optimization removes with certainty — and what constant propagation cannot remove — is the *structural height of the columns*. That is what Optimization 2 exploits.

Optimization 2 — columns, dots, and an exact invariant

The array is $2N$ columns; column k holds n_k dots of weight 2^k . Reduce until no column has more than two.

	in	out	net dots
Full adder	3	1 sum + 1 carry	-1
Half adder	2	1 sum + 1 carry	0

A half adder costs cells and reduces nothing. Minimising half adders is the whole point of Dadda.

This gives an exact invariant, independent of the schedule:

$$\#FA = D_{\text{initial}} - D_{\text{final}}$$

At $N = 32$: $561 - 126 = 435$. The schedule produces exactly 435 — used as the correctness check on the whole algorithm.

Optimization 2 — Wallace and Dadda

Wallace: compress as early as possible. Every group of three dots gets a full adder, at every stage.

Dadda: compress as late as possible. Fix a target height per stage, place only enough compressors to reach it.

$$d_0 = 2, \quad d_{j+1} = \left\lfloor \frac{3}{2} d_j \right\rfloor \implies 2, 3, 4, 6, 9, 13, 19, 28, \dots$$

d_j is the largest height reducible to two in j stages. At $N = 32$ the array has 17 rows, $d_6 = 19 \geq 17$, so **six stages**: $17 \rightarrow 13 \rightarrow 9 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$.

```
function height_dadda(lev : natural) return natural is
  variable temp : integer := 2;
begin
  for i in 0 to lev-1 loop temp := (temp*3)/2; end loop;
  return temp;
end function;
```

Optimization 2 — the schedule is computed, not written

The schedule depends only on the *geometry* of the array, so it can be computed before any signal exists. `constructing_dadda` returns three matrices indexed by (layer, row slot, column):

FA_matrix	the three source slots of every full adder
HA_matrix	the two source slots of every half adder
remaining_system	the dots no compressor consumed

Produced in one pass, kept consistent by mark-and-clear:

```
while node_cnt < 3 loop
  if system(lev)(roww)(col) = '1' then
    node_cnt := node_cnt + 1;
    FA_matrix(lev)(roww)(col) := '1';    -- record the source slot
    system(lev)(roww)(col) := '0';      -- and consume it
  end if;
  roww := roww + 1;
end loop;
```

A consumed dot cannot be assigned twice, and cannot survive into the passed set.

Optimization 2 — the carry accounting

The target height applies to the *next* layer. Column k of layer $\text{lev} - 1$ receives (i) C_{in} carries from column $k - 1$ of the **same** stage, (ii) one sum bit per compressor, (iii) the dots no compressor consumed:

$$C_{\text{in}} + (F + H) + (n - 3F - 2H) = C_{\text{in}} + n - 2F - H \leq d_{\text{lev}-1}$$

A column must be sized against a quantity that **does not exist in its own layer**. Omit C_{in} and the schedule is locally plausible and globally wrong: columns overflow and the array never reaches two rows.

```
count_temp := count + C_out_count;      -- own dots plus carries from col-1
while (count_temp > max_height) loop
  if    count_temp >= (max_height + 2) then count_temp := count_temp - 2; fa_id := fa_id + 1;
  elsif count_temp  = (max_height + 1) then ha_flag := 1; count_temp := count_temp - 1;
  end if;
end loop;
```

Optimization 2 — recovering the port-map positions

Slot order in column k of the next layer is fixed: carries from $k - 1$, then sums, then passed dots.

The problem: a generate statement has no sequential state. Each instance elaborates independently and cannot inherit a running counter from the previous column. Every offset must be recomputable from the matrices alone.

<code>fa_count_col_dadda</code>	full adders in a column
<code>ha_flag_col_dadda</code>	1 if the column has a half adder
<code>num_C_out_from_prev_col</code>	C_{in} , recomputed by counting column $k - 1$
<code>fa_arg_func / ha_arg_func</code>	source row indices of each compressor
<code>remaining_counting_func / remaining_bit_pos</code>	the passed dots and their rows

```
FA_i : FA port map (  
  A => layer(lev)(FA_args(ID)(0))(col),  
  B => layer(lev)(FA_args(ID)(1))(col),  
  Ci => layer(lev)(FA_args(ID)(2))(col),  
  S => layer(lev-1)(ID + C_out_counting)(col),  
  Co => layer(lev-1)(ID)(col+1));
```

Optimization 2 — the schedule is generic in N

N	rows	array dots		stages	Dadda			Wallace
		baseline	optimized		FA	HA	cells	FA instances
8	5	56	45	3	15	9	24	48
16	9	208	153	4	91	21	112	224
32	17	800	561	6	435	45	480	960
64	33	3136	2145	8	1891	93	1984	3968

Per-stage at $N = 32$: 28/92/111/94/53/57 full adders and 12/12/9/6/3/3 half adders.

The Wallace implementation instantiates carry-save adders at the full $2N$ width: 960 full adder instances against 435 FA + 45 HA. Many Wallace instances have constant inputs and are removed by the tool — but constant propagation cannot recover the shorter critical path a column-scheduled tree provides.

Optimization 2 — measured

	Wallace	Dadda	change
Adder cells in the RTL	960 FA	435 FA + 45 HA	−50%
Total cells @ 3.0 ns	4455	3936	−11.7%
Area @ 3.0 ns	6196 μm^2	5725 μm^2	−7.6%
Minimum area	5975 μm^2	5620 μm^2	−5.9%
Min. achieved period	1.87 ns	1.78 ns	−4.8%

The largest single improvement in **delay** of the three optimizations — which is what a shorter, more uniform reduction depth should produce.

Optimization 3 — the cost of the separate negation stage

After Optimization 2 the design is 1180 um^2 larger than the inferred reference. A cell census localises the difference.

@ 3.0 ns	XOR + XNOR	FA cells	INV	total
Structural, Dadda	1230	117	505	3936
Inferred A * B	559	268	311	3035

671 excess XOR cells, and only 117 of 435 scheduled full adders mapped to library adder cells. Both follow from the structure of `mux_and_shift`, which builds a row in two steps — a multiplexer picks A , $2A$ or zero, then every bit is XORed with `sel(2)` to negate. That is $N + 1$ XOR2 per row $\times 16$ rows = 528 cells $\approx 843 \text{ um}^2$ performing nothing but conditional inversion.

The indirect cost is the larger of the two. A full adder is itself XOR-dominated; with an XOR layer feeding the tree, the mapper sees cones that mix negation and compression and decomposes them into generic gates rather than matching `FA_X1`. The low adder count is a consequence of the XOR layer, not an independent effect.

Optimization 3 — merging selection and negation

Two sources, decoded once per row into four mutually exclusive conditions:

$$\begin{aligned}\text{src}_1 &= A_{N-1} \| A (\pm A), & \text{src}_2 &= A \| '0' (\pm 2A) \\ c_{+1} &= \text{sel}(0) \overline{\text{sel}(1)} \overline{\text{sel}(2)}, & c_{-1} &= \text{sel}(0) \overline{\text{sel}(1)} \text{sel}(2), \\ c_{+2} &= \text{sel}(0) \text{sel}(1) \overline{\text{sel}(2)}, & c_{-2} &= \text{sel}(0) \text{sel}(1) \text{sel}(2).\end{aligned}$$

$$q_i = c_{+1} \text{src}_{1,i} + c_{-1} \overline{\text{src}_{1,i}} + c_{+2} \text{src}_{2,i} + c_{-2} \overline{\text{src}_{2,i}}$$

Four terms suffice. The zero row is covered implicitly: all four conditions are false when $\text{sel}(0) = 0$. This is complete provided the encoder never requests a *negated zero* — and the only codes with $\text{sel}(2) = 1$ are 101 and 111, both of which have $\text{sel}(0) = 1$. The property follows from the encoding table; it is not an assumption on it.

Two-level sum of products over four literals $\rightarrow$ one AOI/OAI compound cell per bit. No XOR anywhere. $\overline{\text{src}_1}, \overline{\text{src}_2}$ are the same function of A in all 16 rows, so one set of inverters is shared.

Optimization 3 — measured

@ 3.0 ns	XOR + XNOR	FA cells	AOI/OAI	total	area
Dadda, separate negation	1230	117	1405	3936	5725 μm^2
Dadda, fused selector	85	448	1311	2819	5125 μm^2
Inferred $A * B$	559	268	950	3035	4910 μm^2

	before	after	change
Minimum area	5620 μm^2	5029 μm^2	-10.5%
Minimum achieved period	1.78 ns	1.76 ns	-1.1%

XOR cells 1230 $\rightarrow$ 85. Adder cells 117 $\rightarrow$ 448 — within 7% of the 480 compressors the Dadda schedule actually contains.

The gain is indirect. Removing the XOR layer does not merely delete 528 cells; it lets the mapper match the compressors already present in the RTL.

Verification

One self-checking testbench for all variants, reading NBIT from `common_pkg`:

NBIT	coverage
≤ 8	exhaustive, 65 536 pairs
> 8	corners + 20 000 random

Reference model: `signed(A) * signed(B)` truncated to $2N$ bits, evaluated independently of the design. Mismatches are counted rather than asserted, so a failing variant reports the number of failing vectors.

Exhaustive coverage at $N = 8$ is stronger evidence than random coverage at $N = 32$. The properties under test are all **structural**:

- the recoding table,
- the bias cancellation of Optimization 1,
- the four-term cover of Optimization 3,
- the column schedule of Optimization 2.

Each is a statement about how the array is constructed, not about particular operand values, so each fails identically at both widths.

Each synthesis configuration has a matching testbench configuration, so the binding verified is the binding synthesised.

Library	Nangate 45 nm, typical ECSM
Tool	DC W-2024.09-SP2, compile_ultra
Wire load	5K_hvratio_1_4
Uncertainty	0.05 ns
Input / output delay	0.25 / 0.15 ns, absolute
Driving cell	BUF_X4, load 0.05 pF
Max transition	0.20 ns
Sweep	21 periods, 1.0 to 6.0 ns

Achieved period, not constraint.

$$T_{\text{achieved}} = T_{\text{constraint}} - \text{slack}$$

A violated run still gives a valid netlist, just a slower one. Discarding those would discard the fastest netlists, since the tool only builds aggressive structures under a tight constraint. The identity is exact only because the port delays are absolute rather than a fraction of the period.

compile_ultra is not monotonic. A looser period can give a larger netlist. Verified not to be the area constraint: a controlled run without `set_max_area` gave identical minimum periods and $< 3\%$ area difference, still non-monotonic. All curves are therefore Pareto fronts over the whole sweep.

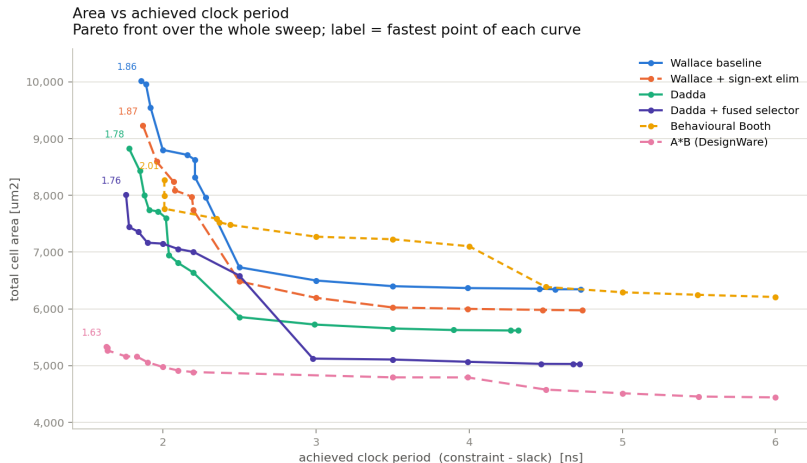
Results — summary

Variant	min. achieved	min. area	cells @ 3.0 ns	gap closed
Wallace, sign-extended (baseline)	1.86 ns	6346 μm^2	4639	—
+ sign-extension elimination	1.87 ns	5975 μm^2	4455	19.5%
+ Dadda reduction	1.78 ns	5620 μm^2	3936	38.1%
+ fused selector	1.76 ns	5029 μm^2	2819	69.1%
Behavioural Booth (adder chain)	2.01 ns	6210 μm^2	4863	7.1%
Inferred A * B	1.63 ns	4441 μm^2	3035	reference

Area gap to the reference closed by 69.1%; delay gap reduced from 0.23 ns to 0.13 ns.

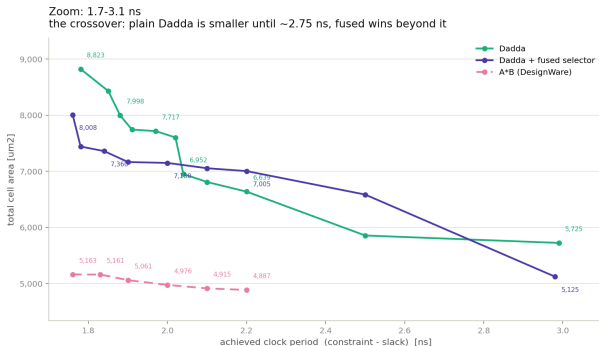
The behavioural variant — a chain of signed additions instead of a tree — is the slowest design in the sweep, but not the largest: the tool builds its own carry-save structures rather than using the instantiated ones. Its *timing* is the informative figure.

Results — the Pareto front



Each curve is the Pareto front of one variant over all 21 runs; dominated points omitted. The ordering holds across the whole range, and the separation is widest at relaxed timing.

Results — a genuine crossing



The two Dadda variants cross **twice**. Near 2.0 ns the variant with separate negation is the smaller of the two.

The fused selector presents deeper compound gates. At intermediate constraints the tool must upsize them to meet timing, and the upsizing costs more than the XOR cells that were removed.

At relaxed timing the upsizing is unnecessary and the fused variant is 591 μm^2 smaller; at tight timing it is faster.

Reproduced across adjacent constraint points, and visible in the census as a shift in *drive strengths* rather than cell counts: an architectural trade-off, not optimizer noise.

Conclusions

Result. 1.76 ns and 5029 μm^2 against 1.63 ns and 4441 μm^2 for an inferred $A * B$ multiplier. 69.1% of the initial area gap recovered.

The three contributions are unequal, and do not follow the size of the structural change each makes.

Sign-extension elimination removes 29.9% of the array but 5.8% of the area — constant propagation removes most of that logic independently of it.

The Dadda schedule gives a comparable area gain and the largest delay gain, consistent with a shorter and more uniform reduction depth.

The fused selector gives the largest area gain, indirectly: not the 528 XOR cells it deletes, but the 331 additional full adders the mapper matches once they are gone.

The general result.

The cost of an RTL construct is set as much by what it prevents the tool from matching as by the gates it implies. One layer of XOR gates between the row generators and the tree accounts for $\approx 600 \mu\text{m}^2$, most of it in the logic behind the layer rather than in the layer itself.

Open.

- Switching activity and power
- Gate-level simulation against SDF
- Sweep below 1.0 ns